Milestone 4 – Group 7

'Imposter Game'

Group Members:

1. Ilia Mosaddegh
2. Xu Chen
3. Alexander Domingo
4. Meyer Kaur Sarna
5. Mishika Bhandari

Our Idea:

We aim to create a Web Game version of the game 'Imposter'. In essence, this will be a 4-player multi-player game that requires players to join a lobby. Three of the four players get the same word and the fourth gets a drastically different one. In each round, each player will have to write one word that describes the word they were given, without giving the real word away to the Imposter. After each round, each player will vote for who they think the imposter is. Correct guesses give one point to the player that made the guess. Incorrect guesses will give a point to the imposter. The imposter's vote will not count towards their score. Players are able play multiple rounds in the same lobby, with new words and a new imposter each time. The origin of this idea came from our team members' recent exposure to the game via social media, and our desire to recreate a game we all enjoy for this class.

Features:

1. **Multi-Player Lobby System**
    - **a.** A player can create a 'Lobby' that their teammates can join using an invite link or a unique code.
    - **b.** Players are able to play multiple consecutive rounds in the same lobby.
    - **c.** **User Stories**:
        - **i.** As a user, I want to be able to join a unique lobby with my friends so we do not have to play with strangers online.
        - **ii.** **Acceptance test**: Players can create new lobbies or join existing ones via a lobby code. A game starts automatically when 4 players are connected. Lobbies persist for multiple rounds with the same players unless someone leaves.
2. **Randomized Role-Assignment System:**
    - **a.** In each round, a randomized algorithm will be implemented to use the unique IDs assigned automatically from the User_Service to randomly decide who will be the Imposter. This leaves it up to chance and removed the risk of predictability.
    - **b.** There are two versions of this system:
        - **i.** **Imposter-Aware**: where the Imposter is aware that they are the imposter, as their assigned word is simply 'Imposter'.
        - **ii.** **Imposter-Unaware:** The Imposter receives a drastically different word.

   c. **User Stories**:
      i. As a user, I want roles and words to change every round to maintain unpredictability.
      ii. **Acceptance test**: Words are drawn randomly from a large dataset or API. Imposter role is assigned randomly and cannot repeat for the same player in consecutive rounds. Mode (aware/unaware) is randomly chosen or selectable before the match.

3. **Clue Submission System**
   a. In each round, each user will be able to submit a one-word clue. The order in which the users are selected is randomzied. Once a clue is submitted, it will automatically be displayed for all to see. There will be a time limit of 30 seconds to submit the clue, otherwis ethe player's opportunity is automatically forfeited.
   b. **User Story:**
      i. As a player, I want to give short related clues about my word so that others can guess or trust me without revealing my exact word.
      ii. **Acceptance Criteria**: Players can type a one-word clue per round. Clues are revealed after all players have submitted. The game advances automatically after all clues are shown.

4. **Voting & Scoring System**
   a. After the clues are submitted, players will have an allotted time period to think and then vote for who they believe the imposter is. Players' scores are tracked across multiple rounds and displayed in a real-time scoreboard to players. The data integrates with the existing User_Service so scores are tied to each user's account.
   b. Scores are based on accuracy
      i. Correct guess → +1 point to player
      ii. Wrong guess → +1 point to imposter (for each wrong voter)
   c. Scores are assigned to the User's unique ID and are accumulated throughout each round.
   d. **User Story:**
      i. As a player, I want to vote for the imposter and earn the correct amount of points for correct guesses so I can fairly compete against others.
      ii. Acceptance criteria: The points are correctly stored, accumulated, and displayed in a scoreboard.

5. **Player Communication System**
   a. This system allows players within the lobby to communicate in between rounds before voting in a 'Discussion' period.
   b. **User Story:** As a player, I want to send short messages or reactions during discussions so I can share suspicions and strategize.
   c. **Acceptance Criteria:** Chat opens only during discussion/voting phases. Messages or preset reactions are visible to all players. Imposter messages appear identical to others to maintain secrecy.

6. **Log-In Page**
   a. Allow users to create an account that has their Name, Email, and Password (using User_Service). Give the option to sign in using their Google and/or Discord Accounts (via their respective APIs).

   b. A unique ID is automatically given (using the existing user sevice)
   c. **User-Story:** I want to create a unique account using my email or discord so that I can log in multiple times with the same credentials.
7. **Local and Global Leadeboard System**
   a. Tracks top players based on total points or win rates, viewable across sessions. Give the option to review results graphically.
   b. **User Story:** As a returning player, I want to see how I rank globally or among friends so that I can track my improvement.
   c. **Acceptance Criteria:** Leaderboard displays usernames, total points, and rank with the option of either within the Lobby or globally.

Analytics:

Types of events to track:

- User lifecycle (registration, login, logout)
- Game flow (room creation, game start/end)
- In-game interaction (asking questions, answering questions, voting)
- UI interaction (button clicks, page views)
- Performance events (load time, API latency)
- Error events (frontend/backend errors)

Event & analytics will store info about game for each round; user active/ inactive, if not used for a long time auto- account deletion - all in user service already

Basic Data Model Design

1. Lobby
- id: int (primary key)
- code: str (6-digit lobby code, unique)
- host_id: int (host user ID)
- status: str ("waiting", "playing", "finished")
- created_at: datetime

2. Game
- id: int (primary key)
- lobby_id: int (foreign key -> Lobby)
- status: str ("waiting", "clue_submission", "voting", "finished")
- current_round: int (current round, 1-3)
- created_at: datetime

3. Player

- id: int (primary key, auto-incrementing)
- game_id: int (foreign key -> Game)
- user_id: int (foreign key -> User)

- is_imposter: bool (is there an insider)
- word: str (assigned word)
- score: int (Score for this round)
- created_at: datetime

4. Clue
- id: int (Primary key)
- game_id: int (Foreign key -> Game)
- player_id: int (Foreign key -> Player)
- round_number: int (Round number 1-3)
- clue_word: str (Clue word)
- submitted_at: datetime

5. Vote
- id: int (Primary key)
- game_id: int (Foreign key -> Game)
- voter_id: int (Voter's Player ID)
- target_id: int (Player ID of the person being voted for)
- created_at: datetime